



Savitribai Phule Pune University
Centre For Distance and Online Education

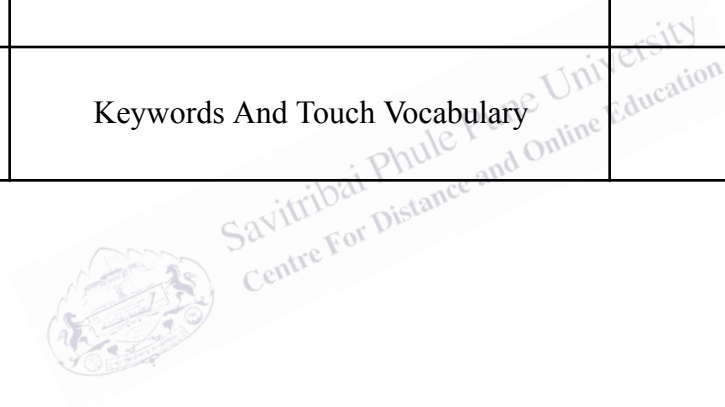
SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application		
Course	Programming from First Principles		
Topic Name	Practical Session Part -3 Merge Sort		
Topic Code	-		
Unit/Module No. & Name	12		Recursion
Self-Learning Hours	-		

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6 - 9
3.0	Nature Or Type	10 - 13
4.0	Examples And Case Studies	14 - 18
5.0	Keywords And Touch Vocabulary	19



OBJECTIVE

1. This unit explains merge sort in simple English with an academic style. The main aim is to help undergraduate students understand what merge sort is, why it is used, and how it works step by step.
2. Merge sort is an important sorting method in computer science. It helps arrange data in increasing or decreasing order in a systematic way. Students often study sorting because ordered data is easier to search, compare, store, and analyze.
3. When data is placed in the correct order, many other tasks become faster and more reliable.
4. After studying this unit, students should be able to define merge sort, describe its main stages, explain its divide-and-conquer nature, and apply the method to simple lists of numbers or words.
5. Students should also learn how merge sort differs from other common sorting methods such as bubble sort, insertion sort, and selection sort. This understanding helps students choose the correct algorithm for different types of problems.
6. Merge sort is part of data structures and algorithms, which is a core area in undergraduate computing courses.
7. It teaches students how an algorithm can solve a large problem by breaking it into smaller parts. This idea appears again in recursion, programming design, database work, artificial intelligence, and software engineering.
8. For this reason, merge sort is more than a single sorting method. It is also a model of clear algorithmic thinking.
9. In practical computing, sorting supports many daily operations. Marks may be sorted by score, names may be sorted alphabetically, products may be sorted by price, and files may be sorted by date.
10. Merge sort is useful when stable and efficient sorting is needed. It is especially valuable when the data set is large and careful performance matters.

1.0 INTRODUCTION

1.1 Why Sorting Matters

1.1.1 Role Of Sorting In Data Work

Sorting is one of the basic operations in computing. Before computers can search, compare, or report information well, the data often needs to be arranged. Imagine a library where books are placed without order, or a contact list where names appear randomly. Finding one item would take more time and more effort. In the same way, computer systems use sorting to make data easier to handle. Sorted data improves speed in many later operations.

1.1.2 Link With Human Reasoning

Human beings also like order. We arrange clothes by type, books by subject, and notes by topic. This natural habit of arranging things helps us think clearly. In computer science, sorting does the same work for digital information. A sorting algorithm gives exact steps that a computer can follow to place data in a chosen order.

1.2 Need For Better Algorithms

1.2.1 Limits Of Simple Methods

At the beginning of algorithm study, students often learn simple methods such as bubble sort or selection sort. These methods are easy to understand, but they may become slow when the amount of data becomes large. A list of five items may be easy to sort by a simple method, but a list of fifty thousand items needs a more efficient approach. This need leads us to improved algorithms such as merge sort.

1.2.2 Value Of Efficiency

Efficiency means doing work in less time or with better use of resources. In algorithm study, efficiency is often discussed in terms of time complexity and space complexity. Merge sort is famous because it gives good time performance on large input sizes. Even though it uses extra memory during merging, its overall structure remains powerful and dependable.

1.3 Core Idea Behind Merge Sort

1.3.1 Divide And Conquer

The heart of merge sort is divide and conquer. This means a large problem is divided into smaller problems, each smaller problem is solved, and then the small solutions are combined into a full solution. This strategy is common in computer science because smaller pieces are often easier to manage. Merge sort takes an unsorted list, divides it into smaller parts until each part is very small, and then merges the parts back in sorted order.

1.3.2 From Disorder To Order

At first, the list may look mixed and confusing. Merge sort does not try to fix the whole list in one step. Instead, it creates order gradually. It sorts small pieces first and then builds the final sorted list by combining those smaller sorted pieces. This systematic design is one reason why merge sort is seen as elegant in academic study.

1.4 Educational Importance

1.4.1 Building Algorithmic Thinking

Merge sort helps students move from basic coding to deeper algorithmic reasoning. It teaches them to think about process, not only output. Students learn that a clever method can reduce unnecessary work. They also learn that a problem can be solved through structure, planning, and repeated steps.

1.4.2 Building Confidence With Recursion

Many versions of merge sort use recursion. Recursion means a function calls itself on smaller inputs. Students may first find recursion difficult, but merge sort provides a clear example of why recursion is useful. The same action is applied again and again until the problem becomes simple enough to solve directly.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Merge Sort

2.1.1 Simple Meaning

Merge sort is a sorting algorithm that divides a list into smaller parts, sorts those parts, and then merges them back together in correct order. The method does not sort by repeated swapping of nearby items. Instead, it creates sorted sublists and then combines them carefully. Because of this design, merge sort performs well for large collections of data.

2.1.2 Everyday Interpretation

A simple way to imagine merge sort is to think of a teacher sorting answer sheets. Instead of trying to place all sheets in order at once, the teacher first divides the stack into smaller piles, arranges each pile, and then combines the piles into one final ordered stack. This picture shows the main idea of the algorithm in an easy way.

2.2 Formal Definition

2.2.1 Standard Definition

Merge sort is a comparison-based sorting algorithm that uses the divide-and-conquer technique. It repeatedly divides an input list into two halves until each part contains one element or no element. Then it merges the parts back together while comparing elements and placing them in sorted order. The result is a complete sorted list.

2.2.2 Technical Features

Merge sort is known for stable behavior and predictable time complexity. A stable sorting algorithm keeps equal elements in the same relative order as they appeared before sorting. This is important when records contain more than one field. For example, if student data is already grouped by class and later sorted by marks, stability helps keep meaningful earlier order among equal marks.

2.3 Main Stages In Definition

2.3.1 Dividing Stage

The first stage is division. The original list is split into two smaller lists. Each smaller list is split again in the same way until very small parts are formed. When a part has only one element, it is already sorted because a single item cannot be out of order with itself.

2.3.2 Merging Stage

The second stage is merging. Two small sorted lists are compared and joined into a larger sorted list. This process continues until all small sorted lists are merged into one final sorted list. The merging stage is the true engine of the method because it creates order by careful comparison.

2.4 Meaning Of Stability

2.4.1 Stable Sorting

A stable sort keeps equal items in their original order. Suppose two students have the same mark, and the earlier list shows A before B. After stable sorting by marks, A should still appear before B. Merge sort can preserve this property when implemented carefully. This is useful in practical data processing.

2.4.2 Why Stability Matters

Stability matters when data has many attributes. In real systems, records may contain name, ID, city, and score. A stable sort allows one level of order to remain while another level is applied. This supports accurate reporting and meaningful data analysis.

2.5 Meaning Of Complexity

2.5.1 Time Complexity

Time complexity describes how the running time grows as the input size becomes larger. Merge sort usually works in $O(n \log n)$ time. This means its growth is better than many simple algorithms on large data. The list is divided in $\log n$ levels, and each level handles about n total comparisons during merging.

2.5.2 Space Complexity

Merge sort generally needs extra space for temporary arrays or lists during merging. Because of this, its space complexity is often $O(n)$. This means it may use more memory than some in-place algorithms. Still, the time efficiency and stable nature often make it a strong choice.

Fig. 1

Merge Sort Process



This visual explains the merge sort workflow, highlighting dividing and merging stages, independent sublist sorting, stability, and its logarithmic time complexity, along with additional space requirements for temporary data storage.

2.6 Comparison With General Sorting

2.6.1 Not Just Rearrangement

Merge sort is not random rearrangement. It is a planned process with a clear structure. Every divide step reduces the problem. Every merge step builds order. This makes the algorithm easier to analyze and teach.

2.6.2 Balance Between Simplicity And Power

Although the full algorithm may seem longer than very basic sorts, its logic is clear once students understand the two repeated actions: divide and merge. This balance between clear structure and strong performance makes merge sort a classic algorithm in academic courses.



3.0 NATURE OR TYPE

3.1 Nature Of Merge Sort

3.1.1 Divide-And-Conquer Nature

The most important nature of merge sort is divide and conquer. The algorithm does not solve the full problem directly. It first breaks the problem into smaller units. These units are easier to sort, and then their solutions are combined. This nature shows how complex tasks can become manageable when divided carefully.

3.1.2 Recursive Nature

Merge sort is commonly recursive. The same sorting action is repeated on the left half and the right half of the list. This repeated structure gives elegance to the algorithm. It also teaches students that a process can call itself until it reaches a simple stopping point, called the base case.

3.2 Structural Nature

3.2.1 Two-Phase Structure

Merge sort has a two-phase structure: dividing and merging. These phases are linked. Dividing creates small sorted units. Merging creates larger sorted units from them. The algorithm moves down the structure while dividing and moves up the structure while merging.

3.2.2 Tree-Like Structure

The process can be shown like a tree. The original list stands at the top. Each split creates branches into smaller lists. The smallest parts appear at the bottom. Then merging combines branches upward until the final sorted list appears at the top again. This tree view helps many students understand the full process.

3.3 Functional Types

3.3.1 Recursive Merge Sort

This is the standard textbook version. It uses function calls that divide the list again and again. It is easy to explain in theory and common in academic teaching. Students learn base case, recursive call, and merge operation through this type.

Fig. 2

Merge Sort's Nature



This diagram illustrates merge sort's core characteristics, including divide-and-conquer strategy, recursion, structured phases, tree-like visualization, extra memory usage, and stability, all contributing to efficient and reliable sorting performance.

3.3.2 Iterative Merge Sort

An iterative version also exists. Instead of calling itself, it starts by merging small groups of size one, then size two, then size four, and so on. This version avoids deep recursion and may be useful in some programming settings. However, the central merging logic remains the same.

3.4 Memory Nature

3.4.1 Not Fully In-Place

Merge sort usually needs extra memory during the merge step. Temporary space stores elements while two sorted parts are being combined. Because of this, merge sort is not usually called a fully in-place sort. This is one of its important practical features.

3.4.2 Controlled Use Of Extra Space

Although it uses extra space, that space serves a clear purpose. It helps merge sorted parts cleanly and correctly. In many systems, the memory cost is acceptable because the gain in order and performance is worth it.

3.5 Stability Nature

3.5.1 Stable By Design

Merge sort can be stable by design if the merge step is handled correctly. When two equal elements are compared, the element from the left side can be chosen first. This preserves earlier order. Stability adds practical value to the algorithm.

3.5.2 Useful In Record Sorting

Stable behavior is useful in sorting records with linked fields. A university database may sort student records by department and then by score. Stability helps preserve one meaningful order while another is applied.

3.6 Performance Nature

3.6.1 Predictable Time Performance

Merge sort gives dependable time performance even in worst-case conditions. Its time complexity remains $O(n \log n)$, whether the list begins in good order, bad order, or mixed order. This predictability is important in algorithm analysis.

3.6.2 Strong Large-Data Behavior

The algorithm is especially strong for large data sets. As the size of data grows, the advantage of efficient algorithms becomes more visible. Merge sort handles growth better than many basic methods that use $O(n^2)$ time.

3.7 Conceptual Types In Teaching

3.7.1 Manual Demonstration Type

Teachers often explain merge sort manually using small lists such as 8, 3, 5, 1, 7, 6. Students split the list into halves, sort each small part, and merge the parts. This teaching type makes the logic visible and supports classroom understanding.

3.7.2 Program Implementation Type

In labs, students implement merge sort in programming languages such as C, C++, Java, or Python. This type focuses on functions, arrays, indices, and temporary memory. Programming practice turns theory into skill.

3.8 Relation To Other Algorithm Types

3.8.1 Internal Sorting

When all data fits in main memory, merge sort acts as an internal sorting method. In such a case, the entire list can be handled directly by the running program. Many classroom examples use this form.

3.8.2 External Sorting Influence

Merge-based ideas are also useful in external sorting, where data is too large to fit in memory at once. In such cases, sorted chunks are created and then merged. This shows that the merge idea scales well beyond simple classroom lists.

4.0 EXAMPLES AND CASE STUDIES

4.1 Basic Worked Example

4.1.1 Sorting A Small List

Consider the list: 8, 3, 5, 1. Merge sort first divides it into two halves: 8, 3 and 5, 1. Then it divides these halves into single elements: 8 and 3, and 5 and 1. Now each single element is already sorted. The algorithm merges 8 and 3 into 3, 8. It merges 5 and 1 into 1, 5. Finally, it merges 3, 8 and 1, 5 into 1, 3, 5, 8. The final list is sorted.

4.1.2 What Students Learn

This small example teaches the complete logic of the algorithm. First, splitting reduces confusion. Second, merging creates order. Third, small sorted parts can build a fully sorted result. The method may look long at first, but it becomes easy after repeated practice.

4.2 Detailed Step Example

4.2.1 Sorting Six Numbers

Take the list: 12, 4, 9, 1, 7, 3. First split it into 12, 4, 9 and 1, 7, 3. Then split again into 12 and 4, 9; and 1 and 7, 3. Split further until single elements appear. Now merge: 4 and 9 become 4, 9. Then 12 and 4, 9 become 4, 9, 12. On the other side, 7 and 3 become 3, 7. Then 1 and 3, 7 become 1, 3, 7. Finally, merge 4, 9, 12 and 1, 3, 7 to get 1, 3, 4, 7, 9, 12.

4.2.2 Academic Note

This example shows that merge sort does not depend on the list being almost sorted. It works through the same pattern each time. This regularity makes analysis easier and reduces the chance of unpredictable behavior.

4.3 Example With Words

4.3.1 Alphabetical Sorting

Merge sort can also sort words. Consider the list: pear, apple, mango, banana. Divide it into pear, apple and mango, banana. Then split into single words. Merge pear and apple into apple, pear. Merge mango and banana into banana, mango. Finally, merge apple, pear and banana, mango

into apple, banana, mango, pear. This shows that merge sort works with any data type that can be compared.

4.3.2 Language Data Use

This example is important because data is not always numerical. Names, file labels, book titles, and city names can also be sorted. Merge sort gives one general method for all comparable data.

4.4 Case Study One

4.4.1 Student Record Management

A college office stores thousands of student records. The staff needs to sort records by roll number before generating reports. A simple algorithm may work on very small lists, but thousands of records require stronger performance. Merge sort becomes a useful option because it offers $O(n \log n)$ time and can preserve stability when equal keys appear.

4.4.2 Institutional Value

In this case, merge sort supports administrative accuracy. Reports become cleaner, searches become easier, and data handling becomes more systematic. The algorithm is not only a classroom topic. It solves real problems in academic institutions.

4.5 Case Study Two

4.5.1 E-Commerce Product Sorting

An online store has many products. Customers want to see items sorted by price, rating, or name. The system may receive large product lists from a database. Efficient sorting improves user experience because ordered results appear faster and more clearly. Merge sort is useful in such a setting, especially where stable sorting helps preserve previous order for equal values.

4.5.2 Business Insight

This case study shows how algorithms affect daily digital life. What seems like a simple sorted product page often depends on well-designed sorting methods behind the screen. Good algorithms support trust, speed, and usability.

4.6 Case Study Three

4.6.1 Sorting In Research Data

A social science researcher collects survey data from many participants. Before analysis, the data must be arranged by region, age, or score. If the data is large, a well-structured sorting method becomes valuable. Merge sort can help prepare the data for later statistical work. Stable ordering is useful if one field must be sorted while another field order remains meaningful.

4.6.2 Research Value

This example shows that sorting is not limited to software companies. It is also important in research, public administration, education, and data-driven decision-making. Algorithms help turn raw information into usable knowledge.

4.7 Comparison Example

4.7.1 Merge Sort And Bubble Sort

Suppose a list has 10,000 items. Bubble sort may require a very large number of repeated comparisons and swaps. Merge sort handles the same list more efficiently through divide and merge steps. The difference becomes more visible as the list grows. This does not mean bubble sort has no teaching value, but merge sort is usually stronger for large data.

4.7.2 Learning Outcome

By comparing algorithms, students learn that correctness is not the only goal in computing. Two algorithms may both produce the same answer, but one may do so much faster. This is why algorithm study includes performance analysis.

4.8 Common Errors In Learning

4.8.1 Confusing Divide With Random Break

Some students think dividing means breaking the list in any random way. In merge sort, the division follows a planned structure, usually into halves. This balanced splitting helps keep the algorithm efficient and conceptually clean.

4.8.2 Forgetting The Merge Logic

Another common mistake is to split correctly but merge carelessly. During merging, students must compare the front elements of the two sorted parts and move the smaller one first. If this rule is ignored, the final list may still be wrong.

4.9 Stepwise Solving Method

4.9.1 Read The Input

First, write the unsorted list clearly. This prevents confusion during repeated splits. Students should label left and right halves if needed.

4.9.2 Divide Systematically

Second, divide the list into halves until single elements remain. At this point, each part is considered sorted.

4.9.3 Merge Carefully

Third, merge the smallest sorted parts into larger sorted parts. Compare one element at a time and place the smaller element first. Continue until one complete sorted list appears.

4.9.4 Verify The Output

Finally, check that every original element appears in the final answer and that the order is correct. This checking step prevents small but common mistakes.

4.10 Programming Perspective

4.10.1 Pseudocode Understanding

Students often study merge sort through pseudocode before full programming. Pseudocode helps them focus on logic rather than language syntax. It usually contains a mergeSort function and a merge function. The first divides and calls itself, while the second combines sorted parts.

4.10.2 Implementation Practice

When implementing the algorithm, students must manage indices, temporary arrays, and function calls carefully. This practice improves coding discipline. It also teaches how theory becomes a working program.

4.11 Advantages And Limits

4.11.1 Main Advantages

Merge sort has several strengths. It offers $O(n \log n)$ time in all major cases, works well on large data, follows a clear structure, and can be stable. It also supports linked-list sorting very effectively in some contexts.

4.11.2 Main Limitation

Its main limitation is extra memory use during merging. For systems with strict memory limits, this may be a concern. Therefore, algorithm choice depends on the problem context, not only on one performance measure.

1. Merge sort divides a large list into smaller parts.
2. Single-element lists are already sorted.
3. The merge step combines sorted parts into a larger sorted list.
4. Merge sort usually runs in $O(n \log n)$ time.
5. The algorithm is stable when equal elements keep their earlier order.
6. Extra memory is often needed during merging.
7. Careful tracing helps students understand each stage.

5.0 Keyword Table

Keyword	Meaning
Merge Sort	A sorting algorithm that divides, sorts, and merges data
Algorithm	A step-by-step method to solve a problem
Sorting	Arranging data in a required order
Divide And Conquer	Solving a large problem by splitting it into smaller parts
Recursion	A process in which a function calls itself
Merge	Combining smaller sorted parts into a larger sorted part
Stable Sort	A sort that keeps equal items in their earlier order
Time Complexity	A measure of how running time grows with input size
Space Complexity	A measure of extra memory used by an algorithm
Comparison	Checking two values to decide their order
Input List	The original unsorted data given to the algorithm
Temporary Array	Extra storage used during merging